LIGO for TypeScript developers on Layer-1

Christian Rinderknecht
Christian.Rinderknecht@trili.tech
TriliTech

17 April 2025

Show and Tell

A Short History of LIGO

- I did my PhD at INRIA under the supervision of Michel Mauny.
- I joined Nomadic in July 2018, where I started designing a smart contract language, similar to Pascal, that would be compiled to Michelson.
- My then-colleague Gabriel Alfour also wanted to make programming smart contracts on Tezos easier, but with a bottom-up approach: building a set of macros that expand into Michelson.
- We joined our efforts in 2019 into what became LIGO.
- I contributed the design and the front-end, he did the type-checker and code generator (50-50 in lines of code).

A Short History of LIGO (continued)

- The idea was for LIGO to offer different concrete syntaxes inspired from mainstream programming languages.
- Those LIGO languages were meant to be domain-specific, but not overly so: supporting special literals (natural numbers, mutez, for examples), but taking most Tezos-specific types and functions from a library.
- The front-end had to lower their input to the common pipeline, while sharing as much code as possible between the processing of the different syntaxes.
- The first syntax was PascaLIGO, as I thought it would suit an imperative style, and lots of keywords would make the code more explicit, given the high stakes.
- Next, turmoil in the nascent ecosystem lead me to design CameLIGO (internally called Ligodity). Then came ReasonLIGO, JsLIGO (which is today's topic), and even PyLIGO, which was never plugged in.

- Nowadays, only two LIGO syntaxes remain: CameLIGO and JsLIGO. The former is a strict subset of OCaml, if we leave aside the Tezos-specific literals, a local type definition construct, and attributes (they occur in prefix position in LIGO).
- JsLIGO started also as a subset but evolved and became syntactically incompatible with its model, TypeScript. For example, a proposal for pattern matching in TypeScript was implemented in JsLIGO.
- The front-end is made, in order, of a preprocessor, a lexer, two parsers and the first tree transformation before type-inference. Each pass can be built as a standalone executable for testing.
- I wrote the preprocessor because, at the time, we had no time to design and implement a module system, together with a build system.
- Unfortunately, it became very popular, even when LIGO got itself a module system, because it enabled conditional compilation (often for new type definitions of the storage, depending on the version of the standard being implemented).

Preprocessor and Lexer

- The preprocessor is not trivial, as it needs to understand UTF-8 encoded glyphs and the kind of comments and strings that depend on the LIGO variant.
- The lexer is also a functor parameterised by lexers that are specific to a given LIGO syntax (comments, strings, keywords etc.). Error reporting has to be aware of linemarkers left by the preprocessor. Some style constraints are enforced (like a linter) for the sake of clarity.
- The lexer is specified as an input to ocamllex.
- Until JsLIGO was added to the party, the lexer signature was a function that returned a single token, which the parser called on demand. Syntactic constructs difficult to parse sometimes rely on lexer hacks, that is, having the parser informing the lexer of the syntactic left-context. Instead, I had the lexer run on all the (preprocessed) input, before applying a series of self-passes on the tokens — injecting virtual tokens when needed to help the parser. This is not as expressive as having the syntactic context, but bi-directional, unbounded lexical context works quite well in practice.

- Perhaps the most interesting self-pass on the tokens is found with JsLIGO:

```
1 :         const f = ([x,y]) ...
```

  could be a parenthesised pair or the start of an arrow function. Analysing the lexical right-context helps disambiguate, and the lexer injects a `ES6FUN` token in front of a function. The parser then knows a function is coming next upon reading the virtual token `ES6FUN`.

- The LIGO pipeline is more than a compiler, because it provides support for a LSP, and that is why the front-end has to keep all the source information, including comments. This adds significant complexity to the lexer, which has to hide it from the parser.

- The two LIGO parsers are generated by Menhir.
- Menhir accepts LR(1) grammars, with semantic actions in OCaml. In practice, many grammars are annotated with precedence declarations to solve LR-conflicts, potentially making the grammars not LR(1), and making it difficult to reason about them.
- Both CameLIGO and JsLIGO are given pure LR(1) grammars, thanks to the rewriting of productions, and the use of virtual tokens.
- Menhir can identify all error states in the LR-automaton, so we can write precise error messages for them. They are precise because the syntactic context is available at the error state, instead of only lexical context, so the past (the intention of the programmer) can be expressed in terms of constructs, and the future too if we take some care when writing the grammar.

Parser and Unified AST

- The parser produces an Abstract Syntax Tree (AST) — named Concrete Syntax Tree (CST) in the code base for legacy reasons.
- The AST is then transformed in a simpler kind of tree, the *unified AST*.
- Whereas the ASTs are specific to each LIGO variant, the unified AST is common to all, and can be considered as the entrance to the common pipeline.